Constructing a Multi-Modal Document & Speech Pipeline -- Starter

You will configure a simulated Phi-4 Multimodal pipeline (text, image, and audio processed together in one call), implement a parallel Whisper baseline with explicit 16kHz resampling, and package both as an Azure ML SDK v2 job -- all runnable with zero GPU, zero live model downloads, and zero external files.

Before you begin: environment setup

In [1]:
```
%pip install -q soundfile datasets librosa azure-ai-ml pillow numpy
```

```
Note: you may need to restart the kernel to use updated packages.
```

Setup: Sample files

Provided for you -- builds the sample image and audio files this activity uses.

In [2]:
```
from PIL import Image, ImageDraw
import numpy as np
import soundfile as sf

img = Image.new("RGB", (600, 400), color=(245, 245, 245))
draw = ImageDraw.Draw(img)
draw.rectangle([40, 40, 560, 360], outline=(0, 0, 0), width=3)
draw.text((60, 60), "SAMPLE DOCUMENT", fill=(0, 0, 0))
draw.text((60, 100), "Invoice #10432", fill=(0, 0, 0))
draw.text((60, 130), "Total Due: $482.00", fill=(0, 0, 0))
img.save("sample_document.jpg")

native_rate = 44100  # deliberately NOT 16kHz, so resampling is genuinely needed
t = np.linspace(0, 2, native_rate * 2)
audio_data = (0.3 * np.sin(2 * np.pi * 220 * t)).astype(np.float32)
sf.write("voice_memo.wav", audio_data, native_rate)

print("SETUP: sample_document.jpg and voice_memo.wav created")
```

```
SETUP: sample_document.jpg and voice_memo.wav created
```

Task 1: Configure the Unified Phi-4 Multimodal Pipeline (10 min)

Complete a function that processes text, image, and audio together in ONE call, using the real `<|image_1|>` / `<|audio_1|>` placeholder token pattern Phi-4 Multimodal uses, and simulates its Mixture-of-LoRAs modality routing conceptually.

Task 1a -- Modality routing (Mixture-of-LoRAs simulation)

Phi-4 Multimodal keeps its language backbone frozen and attaches modality-specific LoRA adapters. Only the adapters for the modalities actually present in the request are switched on.

In [3]:
```python
def route_modality_adapters(has_image, has_audio):
    """Return the modality-specific LoRA adapters that would activate.

    Simulates Phi-4 Multimodal's Mixture-of-LoRAs: the frozen language
    backbone always runs, and a vision or audio adapter is added only
    when that modality is present in the request.
    """
    adapters = []
    if has_image:
        adapters.append("vision_lora")
    if has_audio:
        adapters.append("audio_lora")
    return adapters


# Routing check across every modality combination
for has_image, has_audio in [(False, False), (True, False), (False, True), (True,
True)]:
    print(f"image={has_image!s:<5} audio={has_audio!s:<5} -> "
          f"{route_modality_adapters(has_image, has_audio) or ['(text only - no
adapters)']}")

assert route_modality_adapters(True, True) == ["vision_lora", "audio_lora"]
assert route_modality_adapters(False, False) == []
```

```
image=False audio=False -> ['(text only - no adapters)']
image=True  audio=False -> ['vision_lora']
image=False audio=True  -> ['audio_lora']
image=True  audio=True  -> ['vision_lora', 'audio_lora']
```

Task 1b -- Unified Phi-4 Multimodal call (one prompt, one response)

The prompt carries the real `<|image_1|>` and `<|audio_1|>` placeholder tokens. The function checks that each supplied file has a matching placeholder, loads both inputs, resamples the audio to 16 kHz, routes the adapters, and produces **one** combined response that covers the image and the audio together.

In [4]:
```python
import librosa
import numpy as np
import soundfile as sf
from PIL import Image

TARGET_SR = 16000          # sample rate the audio encoder was trained on
IMAGE_TOKEN = "<|image_1|>"
AUDIO_TOKEN = "<|audio_1|>"


def _count_text_lines(gray, frame_margin=6, ink_threshold=128):
    """Count horizontal bands of dark pixels inside the document frame."""
```

```
    dark = gray < ink_threshold
    rows, cols = np.where(dark)
    top, bottom, left, right = rows.min(), rows.max(), cols.min(), cols.max()
    inner = dark[top + frame_margin:bottom - frame_margin,
                 left + frame_margin:right - frame_margin]
    has_ink = inner.any(axis=1)
    # A new line starts wherever an inked row follows a blank row
    return int(np.sum(has_ink[1:] & ~has_ink[:-1]) + has_ink[0])


def simulate_phi4_multimodal(image_path, audio_path, prompt):
    """Simulated single Phi-4 Multimodal call over text + image + audio.

    Returns (response, active_adapters): ONE combined response string and
    the list of LoRA adapters the request routed to.
    """
    has_image = image_path is not None
    has_audio = audio_path is not None

    # Each supplied modality must have its placeholder token in the prompt
    if has_image and IMAGE_TOKEN not in prompt:
        raise ValueError(f"Image supplied but prompt has no {IMAGE_TOKEN} placeholder")
    if has_audio and AUDIO_TOKEN not in prompt:
        raise ValueError(f"Audio supplied but prompt has no {AUDIO_TOKEN} placeholder")

    # --- Load the image ---
    image = Image.open(image_path).convert("RGB")
    gray = np.asarray(image.convert("L"))
    n_lines = _count_text_lines(gray)

    # --- Load the audio and bring it to 16 kHz ---
    audio, native_sr = sf.read(audio_path, dtype="float32")
    if audio.ndim > 1:                                  # stereo -> mono
        audio = audio.mean(axis=1)
    if native_sr != TARGET_SR:
        audio = librosa.resample(audio, orig_sr=native_sr, target_sr=TARGET_SR)
    duration_s = len(audio) / TARGET_SR
    spectrum = np.abs(np.fft.rfft(audio))
    dominant_hz = np.fft.rfftfreq(len(audio), d=1 / TARGET_SR)[spectrum.argmax()]

    # --- Mixture-of-LoRAs routing ---
    active_adapters = route_modality_adapters(has_image, has_audio)

    # --- Simulated decode: both modalities feed ONE output sequence ---
    description = (
        f"The image ({image.width}x{image.height} px) shows a bordered, single-page "
        f"document with {n_lines} lines of text: a 'SAMPLE DOCUMENT' header, "
        f"Invoice #10432, and a Total Due of $482.00."
    )
    transcript = (
        f"The {duration_s:.2f}-second voice note (resampled {native_sr} Hz -> "
        f"{TARGET_SR} Hz) contains no intelligible speech, only a steady "
        f"~{dominant_hz:.0f} Hz tone, so the transcription is: [tone only - no words]."
    )
    response = f"{description} {transcript}"
    return response, active_adapters


prompt = (
    "<|image_1|>\n<|audio_1|>\n"
    "Describe what the document shows and transcribe the audio note."
)
```

```
response, active_adapters = simulate_phi4_multimodal(
    "sample_document.jpg", "voice_memo.wav", prompt
)

print("PROMPT:\n" + prompt + "\n")
print("UNIFIED RESPONSE (one call, one output):\n" + response + "\n")
print("ACTIVE ADAPTERS:", active_adapters)

# Success-checklist checks
assert IMAGE_TOKEN in prompt and AUDIO_TOKEN in prompt
assert active_adapters == ["vision_lora", "audio_lora"]
assert isinstance(response, str) and "document" in response and "voice note" in response
```

```
PROMPT:
<|image_1|>
<|audio_1|>
Describe what the document shows and transcribe the audio note.

UNIFIED RESPONSE (one call, one output):
The image (600x400 px) shows a bordered, single-page document with 3 lines of text: a
'SAMPLE DOCUMENT' header, Invoice #10432, and a Total Due of $482.00. The 2.00-second
voice note (resampled 44100 Hz -> 16000 Hz) contains no intelligible speech, only a steady
~220 Hz tone, so the transcription is: [tone only - no words].

ACTIVE ADAPTERS: ['vision_lora', 'audio_lora']
```

Success Checklist:

- ☑ The prompt includes both `<|image_1|>` and `<|audio_1|>` placeholder tokens
- ☑ `route_modality_adapters` correctly returns adapter names based on which modalities are present
- ☑ The function returns ONE combined response covering both modalities, not two separate outputs

Task 2: Whisper Baseline with 16kHz Resampling + Azure ML Job (10 min)

Implement a parallel Whisper-style transcription function with explicit resampling, then package both pipelines as an Azure ML SDK v2 job.

Task 2a -- Whisper baseline with explicit 16 kHz resampling

Audio-only baseline. It reads the file's native sample rate and calls `librosa.resample` only when the rate is not already 16 kHz, then builds Whisper's real input: a 30-second window turned into an 80-bin log-Mel spectrogram (400-sample window, 160-sample hop, 3,000 frames).

In [5]:
```
import librosa
import numpy as np
import soundfile as sf

WHISPER_SR = 16000
WHISPER_WINDOW_S = 30
```

```
WHISPER_N_MELS = 80


def simulate_whisper_transcribe(audio_path, target_sr=WHISPER_SR):
    """Simulated Whisper transcription with an explicit 16 kHz resampling step."""
    # Read the audio file at its native rate
    audio, native_sr = sf.read(audio_path, dtype="float32")
    if audio.ndim > 1:                                  # stereo -> mono
        audio = audio.mean(axis=1)

    # Resample only when the native rate is not already 16 kHz
    if native_sr != target_sr:
        audio = librosa.resample(audio, orig_sr=native_sr, target_sr=target_sr)
        step = f"resampled {native_sr} Hz -> {target_sr} Hz with librosa.resample"
    else:
        step = f"already {target_sr} Hz - no resampling needed"
    duration_s = len(audio) / target_sr

    # Whisper front end: pad/trim to a 30 s window -> 80-bin log-Mel spectrogram
    window = np.zeros(WHISPER_WINDOW_S * target_sr, dtype=np.float32)
    window[:min(len(audio), len(window))] = audio[:len(window)]
    mel = librosa.feature.melspectrogram(
        y=window, sr=target_sr, n_fft=400, hop_length=160, n_mels=WHISPER_N_MELS
    )
    log_mel = np.log10(np.maximum(mel, 1e-10))[:, :3000]

    return (
        f"[Whisper baseline] {step}; {len(audio):,} samples = {duration_s:.2f} s at "
        f"{target_sr} Hz; log-Mel input {log_mel.shape[0]}x{log_mel.shape[1]}. "
        f"Transcript (simulated): [no speech detected - steady tone]"
    )


print(simulate_whisper_transcribe("voice_memo.wav"))

# Confirm the conditional: a file that is already 16 kHz is NOT resampled
audio_44k, sr_44k = sf.read("voice_memo.wav", dtype="float32")
sf.write("voice_memo_16k.wav",
         librosa.resample(audio_44k, orig_sr=sr_44k, target_sr=WHISPER_SR), WHISPER_SR)
print(simulate_whisper_transcribe("voice_memo_16k.wav"))
```

```
[Whisper baseline] resampled 44100 Hz -> 16000 Hz with librosa.resample; 32,000 samples =
2.00 s at 16000 Hz; log-Mel input 80x3000. Transcript (simulated): [no speech detected -
steady tone]
[Whisper baseline] already 16000 Hz - no resampling needed; 32,000 samples = 2.00 s at
16000 Hz; log-Mel input 80x3000. Transcript (simulated): [no speech detected - steady
tone]
```

Task 2b -- `pipeline.py`: script simulating both pipelines

Written to `./pipeline_script/`, which becomes the job's code snapshot. The sample files and a conda spec are copied in alongside it so the script has everything it needs when it runs on the cluster.

In [6]:
```
import os
import shutil

os.makedirs("./pipeline_script", exist_ok=True)

PIPELINE_SCRIPT = """\
```

```
# pipeline.py - simulated Phi-4 Multimodal + Whisper benchmark run.
# Entry point for the Azure ML command job. No GPU, no model downloads.
import argparse
import os
import sys

import librosa
import numpy as np
import soundfile as sf
from PIL import Image


def route_modality_adapters(has_image, has_audio):
    adapters = []
    if has_image:
        adapters.append("vision_lora")
    if has_audio:
        adapters.append("audio_lora")
    return adapters


def load_audio(path, target_sr):
    audio, native_sr = sf.read(path, dtype="float32")
    if audio.ndim > 1:
        audio = audio.mean(axis=1)
    if native_sr != target_sr:
        audio = librosa.resample(audio, orig_sr=native_sr, target_sr=target_sr)
        print(f"  resampled {native_sr} Hz -> {target_sr} Hz")
    else:
        print(f"  already {target_sr} Hz - no resampling needed")
    return audio


def run_phi4(image_path, audio_path, target_sr):
    print("[Phi-4 Multimodal] unified call: <|image_1|> + <|audio_1|> + text prompt")
    image = Image.open(image_path).convert("RGB")
    audio = load_audio(audio_path, target_sr)
    adapters = route_modality_adapters(True, True)
    print(f"  image {image.width}x{image.height} px, audio {len(audio) / target_sr:.2f}
s")
    print(f"  active adapters: {adapters}")
    print("  response: invoice #10432, total due $482.00; voice note is a tone with no
speech")
    return adapters


def run_whisper(audio_path, target_sr):
    print("[Whisper baseline] audio-only transcription")
    audio = load_audio(audio_path, target_sr)
    window = np.zeros(30 * target_sr, dtype=np.float32)
    window[:min(len(audio), len(window))] = audio[:len(window)]
    mel = librosa.feature.melspectrogram(
        y=window, sr=target_sr, n_fft=400, hop_length=160, n_mels=80)
    log_mel = np.log10(np.maximum(mel, 1e-10))[:, :3000]
    print(f"  log-Mel input {log_mel.shape[0]}x{log_mel.shape[1]}")
    print("  transcript: [no speech detected - steady tone]")


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--image", default="sample_document.jpg")
    parser.add_argument("--audio", default="voice_memo.wav")
    parser.add_argument("--target_sr", type=int, default=16000)
```

```
    args = parser.parse_args()

    for path in (args.image, args.audio):
        if not os.path.exists(path):
            sys.exit(f"ERROR: input file not found: {path}")

    print("=== Multimodal benchmark job ===")
    run_phi4(args.image, args.audio, args.target_sr)
    run_whisper(args.audio, args.target_sr)
    print("=== PIPELINE COMPLETE: both pipelines ran without error ===")


if __name__ == "__main__":
    main()
"""

CONDA_SPEC = """\
name: multimodal-pipeline-env
channels:
  - conda-forge
dependencies:
  - python=3.10
  - pip
  - pip:
      - numpy
      - soundfile
      - librosa
      - pillow
"""

with open("./pipeline_script/pipeline.py", "w") as f:
    f.write(PIPELINE_SCRIPT)
with open("./pipeline_script/conda.yaml", "w") as f:
    f.write(CONDA_SPEC)

# Ship the sample inputs inside the code snapshot
for sample in ("sample_document.jpg", "voice_memo.wav"):
    shutil.copy(sample, "./pipeline_script/")

print("Code snapshot ./pipeline_script contains:",
sorted(os.listdir("./pipeline_script")))
```

```
Code snapshot ./pipeline_script contains: ['conda.yaml', 'pipeline.py',
'sample_document.jpg', 'voice_memo.wav']
```

Task 2c -- Azure ML SDK v2 job definition

A named `Environment` (CUDA base image + the conda spec above), a named GPU compute target (`gpu-cluster`, scale-to-zero), and a `command()` job that runs `pipeline.py` from the code snapshot. The job is built and inspected locally; the submission lines are left commented out because they need workspace credentials.

In [7]:
```
from azure.ai.ml import command
from azure.ai.ml.entities import AmlCompute, Environment

# Named environment: GPU base image + pinned Python dependencies
env = Environment(
    name="multimodal-pipeline-env",
    description="Phi-4 Multimodal vs Whisper benchmark (simulated)",
    image="mcr.microsoft.com/azureml/openmpi4.1.0-cuda11.8-cudnn8-ubuntu22.04:latest",
```

```
    conda_file="./pipeline_script/conda.yaml",
)

# Named GPU compute target (scales to zero when idle)
gpu_compute = AmlCompute(
    name="gpu-cluster",
    size="Standard_NC4as_T4_v3",
    min_instances=0,
    max_instances=2,
    idle_time_before_scale_down=300,
)

job = command(
    code="./pipeline_script",
    command="python pipeline.py --image sample_document.jpg "
            "--audio voice_memo.wav --target_sr ${{inputs.target_sr}}",
    inputs={"target_sr": 16000},
    environment=env,
    compute=gpu_compute.name,
    display_name="multimodal-vs-whisper-benchmark",
    experiment_name="multimodal-pipeline-evaluation",
    description="Unified Phi-4 Multimodal pipeline vs Whisper baseline",
)

print("Environment name :", job.environment.name)
print("Compute target   :", job.compute)
print("Compute size     :", gpu_compute.size,
      f"(min {gpu_compute.min_instances} / max {gpu_compute.max_instances} nodes)")
print("Code snapshot    :", job.code)
print("Command          :", job.command)

# --- Real submission (requires an Azure ML workspace) ---
# from azure.ai.ml import MLClient
# from azure.identity import DefaultAzureCredential
# ml_client = MLClient(DefaultAzureCredential(), "<subscription-id>",
#                      "<resource-group>", "<workspace-name>")
# ml_client.compute.begin_create_or_update(gpu_compute).result()
# returned_job = ml_client.jobs.create_or_update(job)
# print(returned_job.studio_url)
```

```
Environment name : multimodal-pipeline-env
Compute target   : gpu-cluster
Compute size     : Standard_NC4as_T4_v3 (min 0 / max 2 nodes)
Code snapshot    : ./pipeline_script
Command          : python pipeline.py --image sample_document.jpg --audio voice_memo.wav --target_sr ${{inputs.target_sr}}
```

Task 2d -- Local dry run of `pipeline.py` via subprocess

Runs the script from inside the code snapshot with the same arguments the job uses, so a pass here means the script logic is sound before any real submission.

In [8]:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "pipeline.py",
     "--image", "sample_document.jpg",
     "--audio", "voice_memo.wav",
     "--target_sr", "16000"],
    cwd="./pipeline_script",
```

```
    capture_output=True,
    text=True,
)

print(result.stdout)
if result.stderr.strip():
    print("STDERR:", result.stderr)
print("Exit code:", result.returncode)
assert result.returncode == 0, "pipeline.py failed locally - fix before submitting"
```

```
=== Multimodal benchmark job ===
[Phi-4 Multimodal] unified call: <|image_1|> + <|audio_1|> + text prompt
  resampled 44100 Hz -> 16000 Hz
  image 600x400 px, audio 2.00 s
  active adapters: ['vision_lora', 'audio_lora']
  response: invoice #10432, total due $482.00; voice note is a tone with no speech
[Whisper baseline] audio-only transcription
  resampled 44100 Hz -> 16000 Hz
  log-Mel input 80x3000
  transcript: [no speech detected - steady tone]
=== PIPELINE COMPLETE: both pipelines ran without error ===

Exit code: 0
```

Success Checklist:

- ☑ `simulate_whisper_transcribe` checks the sample rate and resamples with `librosa.resample` when needed
- ☑ The Azure ML `command()` job references a real `pipeline.py` file and a named compute target
- ☑ Running `pipeline.py` locally completes without error

Task 3: Multimodal Architecture Playbook (5 min)

Write a short playbook explaining: why resampling audio is a strict requirement for models like Whisper; how multimodal networks use specialized linear projectors to map visual patches into a shared transformer sequence space; and the cost/latency savings of a single-backbone native architecture versus a legacy multi-model pipeline. (250-400 words)

Multimodal Architecture Playbook

Unified Phi-4 Multimodal vs. the current three-model pipeline

1. Why 16 kHz resampling is a strict requirement

Whisper and Phi-4 Multimodal's audio encoder were both trained on 16 kHz audio. Whisper's front end turns each 30-second window into an 80-bin log-Mel spectrogram using a 400-sample (25 ms) window and a 160-sample (10 ms) hop, which only equal 25 ms and 10 ms at 16,000 samples per second. Feed in our 44.1 kHz voice memo unconverted and nothing crashes, but each frame now covers about 9 ms of real audio, speech is read as 2.76x slower and lower-pitched, and the 30-second window holds only about 10.9 seconds of the recording. The result is a confident but wrong transcript: a silent failure. So both pipelines check the native rate and call `librosa.resample` before either model sees the audio.

2. How linear projectors map visual patches into the shared sequence

The vision encoder (SigLIP in Phi-4 Multimodal) cuts the image into fixed-size patches; a 448×448 crop at 14×14 pixels yields 1,024 patches, and each becomes a feature vector (1,152 dimensions for SigLIP-so400m). Those vectors live in a different space, so a learned projector (a linear layer or small MLP) maps each one to the backbone's hidden size (3,072 for Phi-4-mini). The projected vectors are inserted where the `<|image_1|>` placeholder sits in the prompt, so the transformer attends over image patches, audio frames and text tokens as one sequence. Audio has its own encoder and projector, and the Mixture-of-LoRAs design activates `vision_lora` or `audio_lora` only when that modality is present, leaving the frozen text model unchanged.

3. Cost and latency: one backbone vs. three models

Today a document with a voice note passes through three deployments: captioner, classifier and Whisper. That means three endpoints to keep warm and pay for, three network hops whose latencies add up, and lossy text handoffs, since the classifier never sees the image, only a caption. A single Phi-4 Multimodal endpoint answers the same request in one call and one forward pass, consolidates GPU spend onto one scale-to-zero cluster, and cuts the models the team must version, monitor and secure from three to one. The tradeoff is concentration risk, so Whisper should stay as the accuracy baseline until the benchmark shows the unified model's transcripts are good enough.

Key Points

- Phi-4 Multimodal processes all three modalities in ONE call; Whisper is a specialized, audio-only pipeline requiring its own explicit preprocessing step.
- Resampling is a silent-failure risk, not a crash risk -- always check and resample before passing audio to either model.
- Job packaging logic (Environment, command()) can be fully built and tested locally before any real Azure submission.